LAB 3

BFS & DFS

NAME : ADITYA S V

ROLL NO : CH.SC.U4CSE24256

1.BFS

```
  GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>

#define V 5

// Structure for adjacency list node
struct Node {
    int data;
    struct Node* next;
};

// Graph as adjacency list
struct Node* adj[V];

// Create new node
struct Node* createNode(int v) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = v;
    newNode->next = NULL;
    return newNode;
}

// Add undirected edge
void addEdge(int u, int v) {
    struct Node* newNode = createNode(v);
    newNode->next = adj[u];
    adj[u] = newNode;

    newNode = createNode(u);
    newNode->next = adj[v];
    adj[v] = newNode;
}

// BFS function
void bfs() {
    int visited[V] = {0};
    int queue[V];
```

```
    while (front < rear) {
        int curr = queue[front++];
        printf("%d ", curr);

        struct Node* temp = adj[curr];

        while (temp != NULL) {
            int x = temp->data;

            if (!visited[x]) {
                visited[x] = 1;
                queue[rear++] = x;
            }

            temp = temp->next;
        }
    }
}

int main() {
    // Initialize adjacency list
    for (int i = 0; i < V; i++)
        adj[i] = NULL;

    addEdge(1, 2);
    addEdge(1, 0);
    addEdge(2, 0);
    addEdge(2, 3);
    addEdge(2, 4);

    bfs();

    return 0;
}
```

OUTPUT :

2. DFS

```
  GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>

#define V 5

// Structure for adjacency list node
struct Node {
    int data;
    struct Node* next;
};

struct Node* adj[V];

// Create new node
struct Node* createNode(int v) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = v;
    newNode->next = NULL;
    return newNode;
}

// Add undirected edge
void addEdge(int u, int v) {
    struct Node* newNode = createNode(v);
    newNode->next = adj[u];
    adj[u] = newNode;

    newNode = createNode(u);
    newNode->next = adj[v];
    adj[v] = newNode;
}

// Recursive DFS
void dfsRec(int s, int visited[]) {
    visited[s] = 1;
    printf("%d ", s);
```

GNU nano 7.2

```
// Recursive DFS
void dfsRec(int s, int visited[]) {
    visited[s] = 1;
    printf("%d ", s);

    struct Node* temp = adj[s];

    while (temp != NULL) {
        int i = temp->data;

        if (!visited[i]) {
            dfsRec(i, visited);
        }

        temp = temp->next;
    }
}

// DFS function
void dfs() {
    int visited[V] = {0};

    // handles disconnected graph also
    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            dfsRec(i, visited);
        }
    }
}

int main() {
    // Initialize adjacency list
    for (int i = 0; i < V; i++)
        adj[i] = NULL;

    addEdge(1, 2);
```

```
  GNU nano 7.2
        if (!visited[i]) {
            dfsRec(i, visited);
        }

        temp = temp->next;
    }
}

// DFS function
void dfs() {
    int visited[V] = {0};

    // handles disconnected graph also
    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            dfsRec(i, visited);
        }
    }
}

int main() {
    // Initialize adjacency list
    for (int i = 0; i < V; i++)
        adj[i] = NULL;

    addEdge(1, 2);
    addEdge(1, 0);
    addEdge(2, 0);
    addEdge(2, 3);
    addEdge(2, 4);

    dfs();

    return 0;
}
```

Output :